

PNLIO v9.1

DIRECTOR HUMANO + AGENTE EXPLORADOR CON AUDIO Y EMBEDDINGS (RAG OPTIMIZADO)

Creador: Gonzalo Mauricio de la Rivera Arellano **Fecha:** Julio 2026

Índice

1. Introducción y Características
 2. Requisitos del Sistema
 3. Instalación Paso a Paso
 4. Guía de Uso
 5. Arquitectura Técnica
 6. Solución de Problemas
 7. Código Fuente
-

Introducción

PNLIO v9.1 es un sistema de **interacción humano-AI** que combina:

-  **Grabación y transcripción de audio** en tiempo real
-  **Memoria semántica persistente** usando embeddings (RAG)
-  **Integración con Ollama** para generar respuestas con modelos locales
-  **Contexto dinámico:** Recupera conversaciones pasadas relevantes

Casos de uso: Asistente personal, exploración de conocimiento, documentación de interacciones.



Requisitos del Sistema

Hardware

Componente	Mínimo	Recomendado
CPU	4 núcleos	8+ núcleos
RAM	8 GB	16 GB (para Whisper)
Almacenamiento	10 GB libres	20 GB (para modelos)
Micrófono	Cualquier	Calidad media
GPU	Opcional	NVIDIA (acelera Whisper)

Software

- **Sistema Operativo:** Windows 10+/Linux/macOS
- **Python:** 3.9+ (recomendado 3.10 o 3.11)
- **Ollama:** 0.1+ (debe estar en ejecución)
- **Modelo Ollama:** llama3.2



Instalación Paso a Paso

Paso 1: Instalar Ollama

Windows/macOS: Descargar desde <https://ollama.ai>

Linux:

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Iniciar Ollama:

```
ollama serve # Dejar esta terminal abierta
```

Descargar el modelo:

```
ollama pull llama3.2 # ~4-6 GB
```

Paso 2: Instalar Dependencias de Python

Crear entorno virtual:

```
python -m venv pnlio_env  
source pnlio_env/bin/activate # Linux/macOS  
.\pnlio_env\Scripts\activate # Windows
```

Instalar dependencias:

```
pip install --upgrade pip  
pip install sounddevice scipy numpy sentence-transformers whisper requests
```

Nota: `whisper` tarda varios minutos (descarga 1GB).

Paso 3: Descargar el Código

Opción A: Clonar repositorio (si existe)

```
git clone [URL_DEL_REPOSITORIO]  
cd pnlio_v9.1
```

Opción B: Crear archivo `pnlio.py` con el código de la siguiente diapositiva.

Paso 4: Verificar Instalación

Probar Ollama:

```
curl http://localhost:11434/api/tags
```

Debe devolver `llama3.2` en la lista.

Probar micrófono:

```
import sounddevice as sd
sd.query_devices() # Debe mostrar tu micrófono
```



Guía de Uso

Iniciar PNLIO

```
python pnl.io.py
```

Mensaje de bienvenida:

```
≡ DIRECTOR HUMANO - PNLIO v9.1 ≡
Comandos especiales: 'grabar' para usar voz, 'salir' para terminar.
```

Comandos Básicos

Comando	Descripción	Ejemplo
<code>grabar</code>	Graba 6 segundos de audio	Escribe <code>grabar</code>
<code>salir</code>	Termina la sesión	Escribe <code>salir</code>
<i>Texto</i>	Envía texto directamente a Ollama	“Explica el RAG”

Ejemplo de Flujo

Modo texto:

Director (tú): ¿Qué es el RAG?

Agente: [Respuesta con contexto de memorias previas]

Meta-conciencia: Similitud calculada con éxito. RCR estable.

Modo voz:

Director (tú): `grabar`

[Audio] Grabando 6 segundos... Habla ahora.

[Whisper] Transcribiendo audio...

Transcripción detectada: ¿Cómo funciona la memoria?

Agente: [Respuesta con contexto]

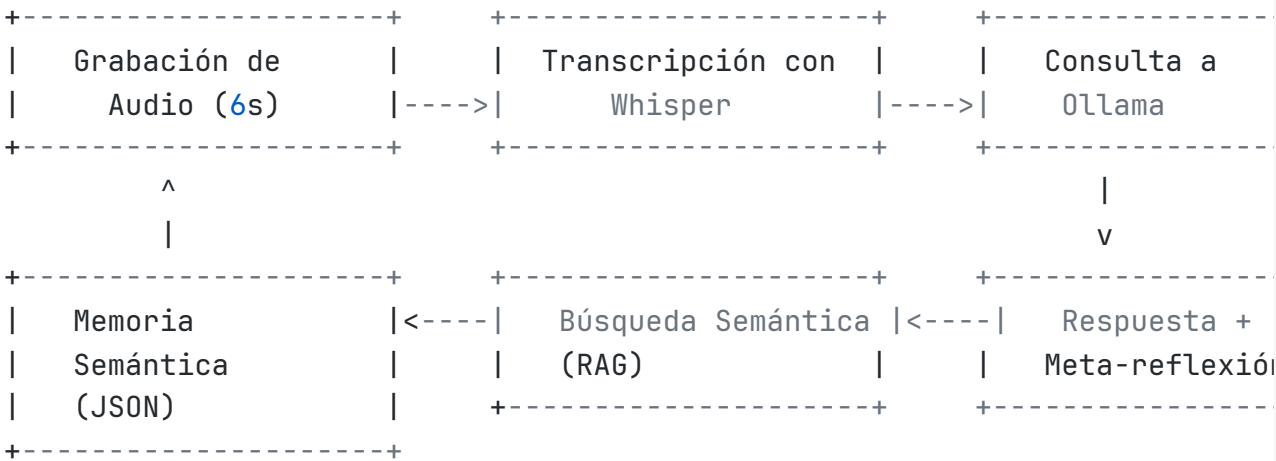
Memoria Semántica

- **Archivo:** `conciencia_semantica.json` (se crea automáticamente)
- **Contenido:** Guarda todas las interacciones con:
 - Fecha y hora
 - Texto de entrada
 - Respuesta del agente
 - Meta-reflexión

- **Búsqueda:** Recupera las 2 interacciones más relevantes semánticamente para enriquecer el contexto.

Arquitectura Técnica

Diagrama de Flujo



Componentes Clave

1. MemoriaEmbeddings

- Modelo: `all-MiniLM-L6-v2` (384 dimensiones, ~80MB)
- Funciones: `cargar()`, `guardar()`, `agregar()`, `buscar_semantica()`
- Usa **similitud de coseno** para búsqueda semántica

2. AgenteDirector

- Métodos: `grabar_audio()`, `transcribir()`, `consultar_ollama()`, `ciclo_director()`

3. Integración con Ollama

- Endpoint: `http://localhost:11434/api/generate`
- Modelo: `llama3.2` (configurable)

Solución de Problemas

Problemas Comunes

Problema	Solución
Ollama no responde	Ejecuta <code>ollama serve</code> en otra terminal.
Modelo <code>llama3.2</code> no encontrado	Ejecuta <code>ollama pull llama3.2</code> .
No se detecta micrófono	Verifica con <code>sd.query_devices()</code> en Python.
Módulo no encontrado (ej: sounddevice)	Ejecuta <code>pip install -r requirements.txt</code> .
Out of Memory (Whisper)	Usa <code>whisper.load_model("tiny")</code> en lugar de <code>"base"</code> .
Archivo <code>conciencia_semantica.json</code> corrupto	Elimina el archivo y reinicia el programa.
ConnectionError en Ollama	Verifica que el puerto 11434 no esté bloqueado por firewall.

Depuración Avanzada

Probar Ollama:

```
curl -v http://localhost:11434/api/generate \
  -d '{"model":"llama3.2","prompt":"Hola","stream":false}'
```

Probar micrófono:

```
import sounddevice as sd
import numpy as np
def callback(indata, frames, time, status):
    print(f"Grabando: {np.max(indata)} dB")
with sd.InputStream(callback=callback):
    input("Presiona Enter para detener...")
```



Código Fuente

```
"""
PNLIO v9.1 - DIRECTOR HUMANO + AGENTE EXPLORADOR CON AUDIO Y EMBEDDINGS (I
Creador: Gonzalo Mauricio de la Rivera Arellano
"""

import logging
import json
from datetime import datetime
from pathlib import Path
import numpy as np
import sounddevice as sd
from scipy.io.wavfile import write
from sentence_transformers import SentenceTransformer
import requests
import whisper

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s')
logger = logging.getLogger(__name__)

OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL = "llama3.2"
WHISPER_MODEL = whisper.load_model("base")

class MemoriaEmbeddings:
    def __init__(self):
        self.embedder = SentenceTransformer('all-MiniLM-L6-v2')
        self.registros = []
        self.embeddings = []
        self.file = Path("conciencia_semantica.json")
        self.cargar()

    def cargar(self):
        if self.file.exists():
            try:
                with open(self.file, "r", encoding="utf-8") as f:
                    data = json.load(f)
                    self.registros = data["registros"]
                    self.embeddings = [np.array(e) for e in data["embeddings"]]
            except Exception as e:
                logger.error(f"Error al cargar la memoria: {e}")
```

```

def guardar(self):
    data = {
        "registros": self.registros,
        "embeddings": [e.tolist() for e in self.embeddings]
    }
    with open(self.file, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def agregar(self, entrada, respuesta, meta_reflex):
    self.registros.append({
        "time": datetime.now().isoformat(),
        "entrada": entrada,
        "respuesta": respuesta,
        "meta": meta_reflex
    })
    emb = self.embedder.encode(entrada + " " + respuesta, normalize_embeddings=True)
    self.embeddings.append(emb)
    self.guardar()

def buscar_semantica(self, query, top_k=2):
    if not self.embeddings:
        return []
    q_emb = self.embedder.encode(query, normalize_embeddings=True)
    scores = [np.dot(q_emb, e) for e in self.embeddings]
    top = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)
    return [self.registros[i] for i in top]

class AgenteDirector:
    def __init__(self):
        self.memoria = MemoriaEmbeddings()

    def grabar_audio(self, segundos=6):
        print(f"\n[Audio] Grabando {segundos} segundos... Habla ahora.")
        fs = 44100
        recording = sd.rec(int(segundos * fs), samplerate=fs, channels=1, dtype='f')
        sd.wait()
        archivo = "input.wav"
        write(archivo, fs, recording)
        print("[Audio] Grabación terminada.")
        return archivo

    def transcribir(self, archivo):

```

```

print("[Whisper] Transcribiendo audio...")
result = WHISPER_MODEL.transcribe(archivo)
return result["text"]

def consultar_ollama(self, prompt):
    try:
        r = requests.post(
            OLLAMA_URL,
            json={"model": MODEL, "prompt": prompt, "stream": False},
            timeout=90
        )
        return r.json()["response"]
    except Exception as e:
        logger.error(f"Error en Ollama: {e}")
        return "Error de conexión con Ollama. Verifica que Ollama esté

def ciclo_director(self):
    print("\n≡ DIRECTOR HUMANO - PNLIO v9.1 ≡")
    print("Comandos especiales: 'grabar' para usar voz, 'salir' para

    while True:
        entrada = input("\nDirector (tú): ").strip()
        if not entrada:
            continue

        if entrada.lower() in ["salir", "exit"]:
            print("Exploración terminada. Memoria semántica resguardada")
            break

        if entrada.lower() == "grabar":
            audio = self.grabar_audio(6)
            entrada = self.transcribir(audio)
            print(f"Transcripción detectada: {entrada}")

        memorias_previas = self.memoria.buscar_semantica(entrada, top.
        contexto = ""
        if memorias_previas:
            contexto = "--- MEMORIAS SEMÁNTICAS RELEVANTES DEL PASADO
            for m in memorias_previas:
                contexto += f"[Registro {m['time'][:10]}]:\n"
                contexto += f"Director dijo: {m['entrada']}\n"
                contexto += f"Agente respondió: {m['respuesta']}\n\n"
            contexto += "-----"

```

```

prompt_final = f"{contexto}Instrucción actual del Director Hu
respuesta = self.consultar_ollama(prompt_final)
meta = f"Reflexión: Similitud calculada con éxito. RCR (Refle:

self.memoria.agregar(entrada, respuesta, meta)

print(f"\nAgente: {respuesta}")
print(f"\nMeta-conciencia: {meta}")
print("\nPropuesta del agente: ¿Hacia qué otra coordenada de

if __name__ == "__main__":
    director = AgenteDirector()
    director.ciclo_director()

```

Anexos

requirements.txt

```

sounddevice=0.4.6
scipy=1.11.4
numpy=1.26.4
sentence-transformers=2.2.2
whisper=1.1.10
requests=2.31.0

```

Configuración Alternativa

- **Whisper más ligero:** `whisper.load_model("tiny")` (~75MB)
- **Embeddings más ligeros:** `SentenceTransformer('paraphrase-MiniLM-L3-v2')` (~40MB)



¿Cómo descargar este documento como PDF?

Opción 1: Desde el navegador (más fácil)

1. Haz clic en el botón  (menú) en la esquina superior derecha del canvas.
2. Selecciona “**Exportar como PDF**” o “**Imprimir**” → “**Guardar como PDF**”.
3. ¡**Listo!** Tendrás un archivo `pnlio_guia.pdf` descargable.

Opción 2: Usando Marp (para mejor formato)

1. Instala Marp:

```
npm install -g @marp-team/marp-cli
```

2. Descarga el contenido del canvas como archivo `.md` (copia el texto).
3. Guárdalo como `pnlio_guia.md` y ejecuta:

```
marp pnlio_guia.md --pdf --output pnlio_guia.pdf
```

Opción 3: Usando Pandoc

```
pandoc pnlio_guia.md -o pnlio_guia.pdf
```



Archivos adjuntos

1. Código Python completo (`pnlio.py`)

Copia el código de la diapositiva anterior y guárdalo en un archivo llamado `pnlio.py` .

2. requirements.txt

Copia este contenido en un archivo llamado `requirements.txt` :

```
sounddevice=0.4.6
scipy=1.11.4
numpy=1.26.4
sentence-transformers=2.2.2
whisper=1.1.10
requests=2.31.0
```



Pasos rápidos para Famae

1. **Descarga el PDF** (usando una de las opciones de arriba).
2. **Sigue las instrucciones de las slides 1-4** (instalación).
3. **Copia el código Python** de la slide “Código Fuente” a un archivo `pnlio.py` .
4. **Ejecuta:**

```
python pnlio.py
```



Soporte

1. **Verifica los logs** en la consola.
2. **Revisa la instalación** de dependencias.
3. **Prueba componentes individualmente** (Ollama, micrófono, Whisper).

¡Gracias por usar PNLIO v9.1!

Desarrollado por: Gonzalo Mauricio de la Rivera Arellano

1. Usa navegador:

- Abre este archivo en un editor de Markdown (ej: VS Code con extensiones de slides).
- Exporta a PDF desde el navegador (Imprimir → Guardar como PDF).

2. Usa Pandoc:

```
pandoc pnllo_guia.md -o pnllo_guia.pdf
```